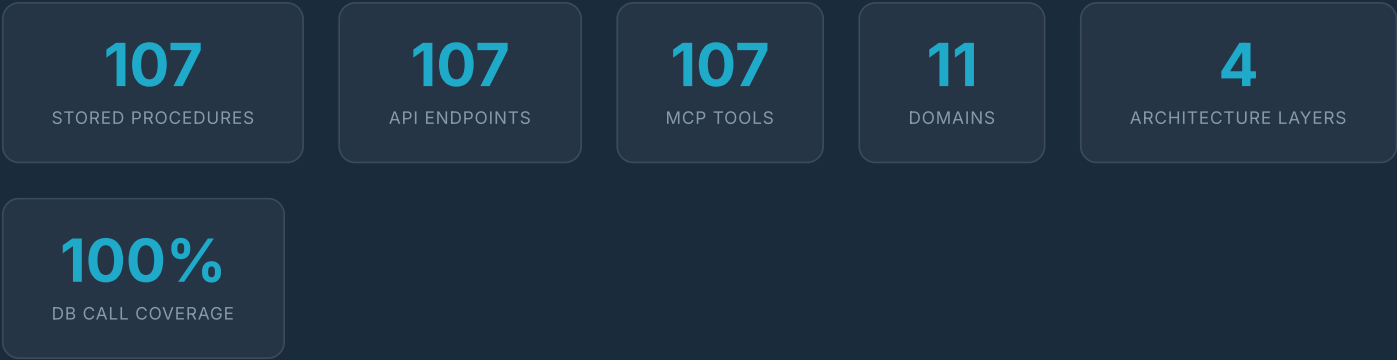


OXPLiveNG Modernization

MCP Server — Client Approval Report

Client	OrthoKinetix (ORX)
System	OXPLiveNG Claims & Billing Dashboard — Modernization to .NET 8 / React 18
Document	MCP Server Architecture & API Coverage — Client Approval
Date	June 2026
Status	Pending Client Approval — Phase B Deliverable
Classification	Confidential — OrthoKinetix / Technijian Use Only



Companion documents
STP-API-MCP Catalog: docs/oxpliveng-sp-api-mcp-catalog.md (11-file series)
System Specification: docs/oxpliveng-mcp-server-specification.md

01 Executive Summary

OXPLiveNG is OrthoKinetix's active claims and billing management system — a SQL Server database with 457 tables, 31 business views, and 11.9 million rows in the Claims History table, currently accessed by a legacy ASP.NET Web Forms application through Entity Framework 6 over raw tables and views with no stored-procedure layer.

This document defines Technijian's plan to build a **four-layer MCP infrastructure** on top of OXPLiveNG that exposes all claims, billing, patient, reconciliation, and warehouse operations to AI agents (including Claude) through the Model Context Protocol — while enforcing strict data-access governance, HIPAA compliance, and a single auditable data path.



What This Enables

- **AI-assisted claims management** — Claude reviews open claims, identifies orphan transactions, and surfaces denial patterns across the AR summary
- **Automated reconciliation** — AI matches ERA 835 payments to claim service lines, flags mismatches, and posts bulk vouchers
- **Billing workflow automation** — AI navigates the Billing Dashboard queue, creates claims from jobs, and corrects service lines before submission
- **Patient data accuracy** — AI reviews insurance coverage, guarantor info, and service lines before claim creation to reduce rejections
- **Admin & compliance operations** — AI manages roles, IP whitelist, groups, and generates audit-ready activity reports

All AI access flows through the MCP server, which calls only the REST API, which calls only stored procedures. **No direct table access from any application layer — ever.** Every call is audited with actor, target, and PHI-redacted details.

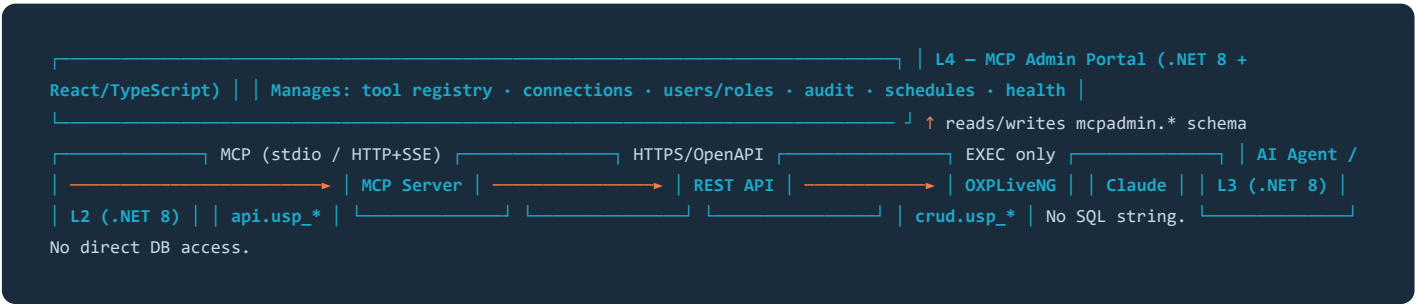
02 Current State — What Needs to Be Built

A direct review of the OXPLiveNG codebase and live database (conducted May 2026) found the following gaps between the current state and the target MCP-ready architecture:

Finding	Evidence	Implication
No business stored-procedure layer	OXPLiveNG has only 6 stored procedures — all are SQL Server auto-generated diagram helpers (sp_creatediagram etc.)	The entire SP encapsulation layer must be built from scratch (~107 SPs)
Direct EF6 / LINQ over tables	Data access uses Entity Framework 6 Database-First (OXPMoel.edmx), primarily direct LINQ over entities	Confirms the SP layer gap — every query bypasses the intended data contract
31 business views carry logic	vw_ClaimDashboard, vw_ARSummaryPRBalance, vw_ClaimTransaction, 8 Notification views, 3 chetuSA views	These are the primary read targets for Phase 1 SP wrapping — high-value, well-tested view logic
Credentials hardcoded	SQL login chetuSA with plaintext password in Web.config:26-27	Must be remediated before Phase 3 — service account with Azure Key Vault reference only
Schema noise	457 tables including *_Backup_*, Temp_*, and *_Consolidated duplicates	The SP/API layer provides a clean, versioned contract — backup/temp tables excluded by config-driven allow-list
No MCP server or REST API	No /api project, no OpenAPI spec, no MCP tooling exists	All four layers (DB SPs → REST API → MCP Server → Admin Portal) must be built

03 Architecture — Four-Layer Design

The OXPLiveNG MCP infrastructure follows a strict four-layer design. This is the same reference architecture used for the existing orx-mcp-daisybill1837 integration, ensuring consistent operations, deployment (IIS), and developer onboarding across all ORX MCP servers.



Layer	Tech Stack	Responsibility
L1 — Database SP Layer	T-SQL on OXPLiveNG · api schema (reads) · crud schema (writes) · SSDT migrations	All reads/writes encapsulated as stored procedures. api.* wraps 31 views; crud.* provides full CRUD per core entity. Zero inline SQL anywhere above this layer.
L2 — REST API	ASP.NET Core 8 Web API · Dapper · OpenAPI/Swagger · Microsoft.Data.SqlClient	Calls SPs only; exposes OpenAPI contract; handles auth (Entra ID), validation, paging, and PHI-redacted audit logging. Routes are generated from the DB catalog — adding an SP adds an endpoint without a code deploy.
L3 — MCP Server	.NET 8 · Official ModelContextProtocol C# SDK · stdio + HTTP/SSE transports	Exposes MCP tools derived from the OpenAPI catalog. Has no SQL connection string. Calls L2 only. Tool definitions, rate limits, and feature flags are all DB-resident configuration.
L4 — Admin Portal	ASP.NET Core 8 + React/TypeScript · Entra SSO · mcpadmin.* schema	Manages the MCP server without code changes: enable/disable tools, manage connections, view audit logs, configure schedules, set retention policies, run test queries. Mirrors OrxDaisyBillMcp.admin.

Key principle (non-negotiable): The MCP Server (L3) has no `Microsoft.Data.SqlClient` reference — it cannot access the database directly. All data flows through L2. This is enforced at the project level and verified by CI.

04 Domain Coverage Matrix

The table below lists all 11 business domains covered by the OXPLiveNG MCP server, with stored procedure, API endpoint, and MCP tool counts per domain. Every database operation in OXPLiveNG is reachable through this contract.

107 STORED PROCEDURES		107 API ENDPOINTS		107 MCP TOOLS		11 DOMAINS
#	Domain	STPs	APIs	Tools	What It Covers	
1	Claims	25	25	25	Claims dashboard, AR summary, transaction management, orphan matching, duplicate detection	
2	Patient / Account	25	25	25	Patient demographics, insurance, medical info, service lines, guarantor, health plans, documents	
3	Billing Dashboard	3	3	3	Billing job queue, patient info panel, document file-type resolution	
4	Admin & Security	22	22	22	Roles, permissions, users, organizations, groups, IP whitelist, billing codes	
5	Authorization	2	2	2	PA request pre-population — patient and order details for prior authorization forms	
6	Documents (Core)	1	1	1	System-wide document list for document management and attachment workflows	
7	Groups / Risk Payers	5	5	5	Master groups, risk payer group configuration and assignments	
8	Orders	5	5	5	Order listing, detail, fulfillment tracking, cancellation, authorization status	
9	Reconciliation	8	8	8	ERA/EOB payment vouchers, orphan matching, monthly close reporting, bulk ERA import	
10	Warehouse / Inventory	8	8	8	Warehouse locations, product catalog, stock management, inter-location transfers	
11	Document Storage (AWS/S3)	3	3	3	Patient file metadata after S3 upload — create, update, delete document records	
TOTAL		107	107	107	100% of OXPLiveNG database operations	

05 Detailed Capability Reference

Every stored procedure in the OXPLiveNG MCP server is listed below, grouped by domain. Each row shows whether the operation reads data or modifies it, the API endpoint it maps to, and a plain-English description of what billing staff and AI assistants can do with it. Total: **60 read operations, 47 write operations.**

01 Claims17 read · 8 write				
#	Stored Procedure	R/W	API Endpoint	What It Does
1.1	api.usp_Claim_Dashboard_Get	READ	GET /api/v1/claims/dashboard	Paged claims dashboard with filtering by patient name, claim number, date range, status, or payer. Returns header rows with payer, status, and financial summary.
1.2	api.usp_AR_Summary_Get	READ	GET /api/v1/claims/ar-summary	Rolled-up accounts receivable summary showing outstanding balances by payer and aging bucket. Core AR follow-up and denial management tool.
1.3	api.usp_AR_SummaryByPayerGroup_Get	READ	GET /api/v1/claims/ar-summary/by-payer-group	AR Summary filtered to a specific Risk Payer Group. Lets managers analyze collections performance against a contracted group.
1.4	api.usp_ClaimTransaction_Unreconciled_List	READ	GET /api/v1/claims/transactions/unreconciled	ERA/payment transactions imported but not yet matched to a claim service line. Used in reconciliation to identify open transactions.
1.5	api.usp_ClaimTransaction_Reconciled_List	READ	GET /api/v1/claims/transactions/reconciled	Payment transactions already matched to claim service lines. Used for

#	Stored Procedure	R/W	API Endpoint	What It Does
				audit trails, dispute resolution, and confirming payment posting accuracy.
1.6	<code>api.usp_ClaimTransaction_ForService_List</code>	READ	GET <code>/api/v1/claims/services/{serviceId}/transactions</code>	All transactions (reconciled and unreconciled) attached to a single claim service line. Shown in service-level detail panel.
1.7	<code>crud.usp_ClaimTransaction_Update</code>	WRITE	PUT <code>/api/v1/claims/transactions/{id}</code>	Updates an existing claim transaction record — corrects payment amount, check number, or reconciliation status after payer correction.
1.8	<code>crud.usp_ClaimTransaction_Delete</code>	WRITE	DELETE <code>/api/v1/claims/transactions/{id}</code>	Soft-deletes a claim transaction posted in error — duplicate ERA import, wrong payer, or test transaction.
1.9	<code>api.usp_Claim_ServiceList_ByClaimId</code>	READ	GET <code>/api/v1/claims/{claimId}/services</code>	All service lines on a claim — procedure codes, modifiers, billed/paid amounts, and status per line.
1.10	<code>api.usp_Claim_ServiceList_ByJobId</code>	READ	GET <code>/api/v1/claims/services/by-job/{jobId}</code>	Service lines linked to a Job (order). Pre-populates service line details from the job during billing workflow.
1.11	<code>api.usp_Claim_ServiceList_ByClaimRef</code>	READ	GET <code>/api/v1/claims/services/by-ref/{claimRef}</code>	Service lines by external claim reference number (payer or clearinghouse). Used in ERA reconciliation.
1.12	<code>api.usp_OrphanTransaction_List</code>	READ	GET <code>/api/v1/claims/orphan-transactions</code>	Unmatched ERA payment transactions not linked to any claim service line. Resolution

#	Stored Procedure	R/W	API Endpoint	What It Does
				requires manual matching or write-off.
1.13	api.usp_OrphanTransaction_MismatchFiltered_List	READ	GET /api/v1/claims/orphan-transactions/mismatch	Orphan transactions where paying payer doesn't match claim payer — indicates routing errors or cross-payer postings.
1.14	crud.usp_OrphanTransaction_Delete	WRITE	DELETE /api/v1/claims/orphan-transactions/{id}	Removes an orphan transaction that is a duplicate, test record, or otherwise invalid. Operator role or higher required.
1.15	crud.usp_OrphanTransaction_Process	WRITE	POST /api/v1/claims/orphan-transactions/{id}/process	Matches an orphan transaction to a claim service line and posts payment atomically. Inserts ClaimTransaction and deactivates orphan.
1.16	crud.usp_OrphanTransaction_Transfer	WRITE	POST /api/v1/claims/orphan-transactions/{id}/transfer	Transfers an orphan transaction to a different claim when ERA payment was posted under the wrong claim number.
1.17	crud.usp_Claim_WithServices_Create	WRITE	POST /api/v1/claims	Creates a new claim header with one or more service lines in an atomic operation. Used in Billing for new claims not from orders.
1.18	api.usp_Claim_LevelData_ByClaimRef	READ	GET /api/v1/claims/by-ref/{claimRef}/detail	Claim-level header data (patient, payer, dates, provider) for Claims Dashboard detail panel by claim reference.
1.19	api.usp_ClaimService_SummaryById	READ	GET /api/v1/claims/services/{claimServiceId}/summary	Single service line summary with aggregated payment, adjustment, and

#	Stored Procedure	R/W	API Endpoint	What It Does
				balance figures. Shown in service-level detail panel.
1.20	api.usp_Claim_PayerList_Get	READ	GET /api/v1/claims/payers	List of payers with Risk Payer Group names. Populates payer filter dropdowns and group-based reporting.
1.21	crud.usp_Claim_PayerGroup_Upsert	WRITE	POST /api/v1/claims/payers/group	Creates or updates payer-to-group mapping. Assigns payers to Risk Payer Groups for AR reporting and filtering.
1.22	crud.usp_Claim_EditAndNote_Save	WRITE	POST /api/v1/claims/{claimId}/edit	Saves edits to claim header (user, provider, risk payer, status, submit date) and appends status note atomically.
1.23	api.usp_Claim_OpenList_Get	READ	GET /api/v1/claims/open	All open (unresolved, not closed/paid-in-full) claims. Gives billing staff an actionable queue.
1.24	api.usp_Claim_History_List	READ	GET /api/v1/claims/{claimId}/history	Full audit and note history for a claim — all status changes, notes, and system events in chronological order.
1.25	api.usp_Claim_Duplicates_List	READ	GET /api/v1/claims/duplicates	Claims identified as potential duplicates — same patient, payer, service date, and procedure code submitted multiple times. Used by billing QA.

02 Patient / Account

14 read · 11 write

#	Stored Procedure	R/W	API Endpoint	What It Does
2.1	api.usp_Patient_Search	READ	GET /api/v1/patients/search	Patient search by name, DOB, phone,

#	Stored Procedure	R/W	API Endpoint	What It Does
				address, or health plan. Returns matching records with demographic and insurance snapshot.
2.2	api.usp_Patient_Detail_GetById	READ	GET /api/v1/patients/{patientJobId}	Full patient record with all demographic fields and primary health plan. Used on patient detail and edit pages.
2.3	crud.usp_Patient_Insert	WRITE	POST /api/v1/patients	Creates new patient record (PatientJob) and optionally links to group. Foundational create for new patient intake.
2.4	crud.usp_Patient_Update	WRITE	PUT /api/v1/patients/{patientJobId}	Updates existing patient's demographic and insurance information from the Patient Edit form.
2.5	crud.usp_Patient_SoftDelete	WRITE	DELETE /api/v1/patients/{patientJobId}	Soft-deletes patient (sets PStatus=true). Removes from active lists while preserving all historical claims for compliance.
2.6	api.usp_Patient_MedicalInfo_GetByJobId	READ	GET /api/v1/patients/{jobId}/medical-info	Injury details, body parts, and diagnosis codes for patient job. Drives the medical info section of Billing Dashboard.
2.7	crud.usp_Patient_MedicalInfo_Update	WRITE	PUT /api/v1/patients/medical-info/{id}	Updates medical info fields (injury date, body part, diagnosis). Called from Billing Dashboard medical info edit panel.
2.8	api.usp_Patient_InsuranceInfo_GetByJobId	READ	GET /api/v1/patients/{jobId}/insurance	Patient's insurance records (primary, secondary, tertiary) — payer name, member ID, group number, copay/deductible.
2.9	crud.usp_Patient_InsuranceInfo_Update	WRITE	PUT /api/v1/patients/insurance/{id}	Updates insurance record when patient's payer, member ID, or coverage dates

#	Stored Procedure	R/W	API Endpoint	What It Does
				change. Critical for clean claim submission.
2.10	api.usp_Patient_JobServiceLine_GetByJobId	READ	GET /api/v1/patients/{jobId}/service-lines	Product/service lines on patient job — HCPCS codes, modifiers, quantities, and prices that drive claim creation.
2.11	crud.usp_Patient_JobServiceLine_Upsert	WRITE	POST /api/v1/patients/{jobId}/service-lines	Creates or updates a service line on patient job when billing staff adds or corrects a procedure before claim submission.
2.12	crud.usp_Patient_JobServiceLine_Delete	WRITE	DELETE /api/v1/patients/service-lines/{id}	Soft-deletes a service line from patient job when billing staff removes a procedure entered in error.
2.13	api.usp_Patient_GuarantorInfo_GetByJobId	READ	GET /api/v1/patients/{jobId}/guarantor	Guarantor (responsible party) information — name, relationship, contact, address. Required for patient-responsibility billing.
2.14	crud.usp_Patient_GuarantorInfo_Update	WRITE	PUT /api/v1/patients/guarantor/{id}	Updates guarantor contact and demographic info when responsible party changes (e.g. patient turns 18, divorce).
2.15	api.usp_Patient_HealthPlan_GetById	READ	GET /api/v1/patients/{patientJobId}/health-plans	All health plan records for patient — plan name, effective/term dates, copay, deductible. Drives the health plan section.
2.16	crud.usp_Patient_HealthPlan_Insert	WRITE	POST /api/v1/patients/{patientJobId}/health-plans	Adds one or more health plan records during new patient intake or plan change workflow.
2.17	crud.usp_Patient_HealthPlan_Delete	WRITE	DELETE /api/v1/patients/health-plans/{healthPlanId}	Deletes a health plan record when added in error or patient terminates a secondary plan.
2.18	api.usp_Patient_Claim_InfoById	READ	GET /api/v1/patients/{patientJobId}/claims/{orderId}	Patient's claim information for a specific order —

#	Stored Procedure	R/W	API Endpoint	What It Does
				claim header, payer info, and attached product/procedure details.
2.19	<code>api.usp_Patient_OrderHistory_List</code>	READ	GET <code>/api/v1/patients/{patientJobId}/orders</code>	Order history — all orders, statuses, products, authorization info, and assigned staff. Shown in Patient Orders tab.
2.20	<code>api.usp_Patient_ClaimHistory_List</code>	READ	GET <code>/api/v1/patients/{patientJobId}/claim-history</code>	Claim history for patient — all submitted claims, statuses, and dates. Shows the patient's complete claim lifecycle.
2.21	<code>api.usp_Patient_Eligibility_GetByServiceType</code>	READ	GET <code>/api/v1/patients/{patientJobId}/eligibility/plan</code>	Health plan record for a specific service type. Pre-populates eligibility request form.
2.22	<code>api.usp_Patient_Documents_List</code>	READ	GET <code>/api/v1/patients/{patientJobId}/documents</code>	All uploaded documents for patient — EOBs, authorization letters, insurance cards, referrals. Used in Patient Documents tab.
2.23	<code>api.usp_Patient_NotificationPayMetrics_Get</code>	READ	GET <code>/api/v1/patients/notification/pay-metrics</code>	Payer payment metrics — average/max first and last payment days, 15 and 22-day payment sums. Used for notification thresholds.
2.24	<code>api.usp_Dropdown_References</code>	READ	GET <code>/api/v1/dropdowns/{type}</code>	Nine type-ahead/select dropdown endpoints: Practitioner, Physician, Stat, Diagnosis, Modifier, AtRiskPayer, HCPCS, HealthPlan.
2.25	<code>crud.usp_Claim_FromBilling_Insert</code>	WRITE	POST <code>/api/v1/billing/claims/from-job</code>	Creates a claim from a billing job — the core action when billing staff submits a claim. Bridges job/order world to claims world.

#	Stored Procedure	R/W	API Endpoint	What It Does
3.1	<code>api.usp_Billing_Dashboard_Get</code>	READ	GET <code>/api/v1/billing/dashboard</code>	Billing Dashboard grid — primary workspace for billing staff to view all patient jobs requiring claim preparation, by status.
3.2	<code>api.usp_Billing_PatientInfo_GetByJobId</code>	READ	GET <code>/api/v1/billing/{jobId}/patient-info</code>	Patient demographic snapshot for job shown in Billing Dashboard detail panel. Pre-populates billing form header.
3.3	<code>api.usp_Billing_FileTypeName_GetByFileId</code>	READ	GET <code>/api/v1/billing/file-types/{fileId}</code>	Resolves file type ID to a human-readable label (e.g. 'Authorization Letter', 'EOB'). Used in document chip rendering.

#	Stored Procedure	R/W	API Endpoint	What It Does
4.1	<code>api.usp_Admin_Role_List</code>	READ	GET <code>/api/v1/admin/roles</code>	All application roles (excluding Super Admin). Drives Role Management grid in Admin portal.
4.2	<code>crud.usp_Admin_Role_CreatePermission</code>	WRITE	POST <code>/api/v1/admin/roles/{roleId}/permissions</code>	Creates permission assignments for role — maps role to page permissions (read/create/edit/delete flags per page).
4.3	<code>crud.usp_Admin_Role_Delete</code>	WRITE	DELETE <code>/api/v1/admin/roles/{roleId}</code>	Deletes an application role. Requires no users currently assigned to it.
4.4	<code>api.usp_Admin_Role_PermissionList</code>	READ	GET <code>/api/v1/admin/roles/{roleId}/permissions</code>	Full permission matrix for role — each application page with current read/create/edit/delete flags.
4.5	<code>crud.usp_Admin_Role_Update</code>	WRITE	PUT <code>/api/v1/admin/roles/{roleId}</code>	Updates role's name, user type, and permission assignments atomically from the Edit Role save action.
4.6	<code>api.usp_Admin_User_GetByAspNetUserId</code>	READ	GET <code>/api/v1/admin/users/by-aspnet-id/{id}</code>	User profile — name, email, role, group, last login, org. Core user lookup for Admin User Management grid.
4.7	<code>crud.usp_Admin_User_ProfileUpdate</code>	WRITE	PUT <code>/api/v1/admin/users/{aspNetUserId}/profile</code>	Updates user profile — name, phone, title, org assignment. Called from Admin User Edit or user's own profile page.
4.8	<code>crud.usp_Admin_User_EmailUpdate</code>	WRITE	PUT <code>/api/v1/admin/users/{userId}/email</code>	Updates user's email address. Separate from profile update because email change requires ASP.NET Identity sync.
4.9	<code>crud.usp_Admin_User_RoleUpdate</code>	WRITE	PUT <code>/api/v1/admin/users/{userId}/role</code>	Changes user's assigned role when promoting, demoting, or transferring a user in Admin User Management.

#	Stored Procedure	R/W	API Endpoint	What It Does
4.10	api.usp_Admin_User_Roles_List	READ	GET /api/v1/admin/roles/assignable	All assignable roles for user assignment dropdown. Used in Admin User Create and Edit forms.
4.11	api.usp_Admin_SecurityQuestion_GetByEmail	READ	GET /api/v1/admin/users/security-questions	Security questions configured for user by email. Used in Forgot Password / account recovery flow.
4.12	api.usp_Admin_Organization_List	READ	GET /api/v1/admin/organizations	All organizations configured in system. Used in Admin Organization Management grid and user/group assignment dropdowns.
4.13	crud.usp_Admin_Organization_Upsert	WRITE	POST /api/v1/admin/organizations	Creates or updates organization record from the Admin Organization form.
4.14	api.usp_Admin_Group_List	READ	GET /api/v1/admin/groups	All user groups with filter support. Groups are the primary unit for assigning patients, orders, and permissions.
4.15	crud.usp_Admin_Group_Upsert	WRITE	POST /api/v1/admin/groups	Creates or updates a group from the Admin Group Create and Edit forms.
4.16	crud.usp_Admin_Group_Delete	WRITE	DELETE /api/v1/admin/groups/{groupId}	Permanently deletes group and all its member associations when the group is no longer needed.
4.17	crud.usp_Admin_Group_MemberInsert	WRITE	POST /api/v1/admin/groups/{groupId}/members	Adds a user to a group when assigning new staff for patient/order access.
4.18	api.usp_Admin_WhitelistIP_List	READ	GET /api/v1/admin/security/whitelist-ips	All whitelisted IP addresses allowed to access the system. Used in Admin Security IP Whitelist management page.
4.19	crud.usp_Admin_WhitelistIP_Upsert	WRITE	POST /api/v1/admin/security/whitelist-ips	Adds or updates a whitelisted IP when adding an office location or updating a description.
4.20	crud.usp_Admin_WhitelistIP_Delete	WRITE	DELETE /api/v1/admin/security/whitelist-ips/{id}	Removes IP from whitelist when an office location closes or IP range changes.
4.21	api.usp_Admin_BillingCode_List	READ	GET /api/v1/admin/billing-codes	All billing codes configured in the system. Used in Admin Billing Code Management grid.
4.22	crud.usp_Admin_BillingCode_Delete	WRITE	DELETE /api/v1/admin/billing-codes/{id}	Removes a billing code when it is retired or entered in error.

#	Stored Procedure	R/W	API Endpoint	What It Does
5.1	api.usp_Authorization_PatientDetails_Get	READ	GET /api/v1/authorization/patient/{patientJobId}/details	Patient and insurance details needed to complete an order authorization request form. Pre-populates PA request to payers.
5.2	api.usp_Authorization_OrderDetails_Get	READ	GET /api/v1/authorization/order/{orderId}/details	Order details for PA request — products, HCPCS codes, quantities, physician info. Complete prior authorization dataset.

06 Documents (Core)

1 read · 0 write

#	Stored Procedure	R/W	API Endpoint	What It Does
6.1	api.usp_Document_List	READ	GET /api/v1/documents	All documents accessible to requesting user. Used on Documents management page and document attachment workflows.

07 Groups / Risk Payers

2 read · 3 write

#	Stored Procedure	R/W	API Endpoint	What It Does
7.1	api.usp_Group_MasterGroup_List	READ	GET /api/v1/groups/master	All master groups — top-level organizational groupings containing sub-groups and users.
7.2	crud.usp_Group_MasterGroup_Upsert	WRITE	POST /api/v1/groups/master	Creates or updates a master group from the Master Group Create and Edit forms.
7.3	crud.usp_Group_MasterGroup_Delete	WRITE	DELETE /api/v1/groups/master/{id}	Deletes a master group from the Master Group Management admin page.
7.4	api.usp_Group_RiskPayer_GetByGroupId	READ	GET /api/v1/groups/{groupId}/risk-payers	List of risk payers assigned to a group. Used in Group Risk Payer configuration page.
7.5	crud.usp_Group_RiskPayer_Save	WRITE	PUT /api/v1/groups/{groupId}/risk-payers	Replaces full risk payer list for group (delete existing, insert new). Called when administrator updates tracked payers.

08 Orders

2 read · 3 write

#	Stored Procedure	R/W	API Endpoint	What It Does
8.1	api.usp_Order_List	READ	GET /api/v1/orders	All orders in system. Used by Order Management grid for warehouse staff, coordinators, and billing staff.
8.2	api.usp_Order_GetById	READ	GET /api/v1/orders/{orderId}	Full order detail including line items, authorization, diagnosis, and documents. Used in order detail views.
8.3	crud.usp_Order_DispatchReceived_Update	WRITE	PUT /api/v1/orders/{orderId}/fulfillment-dates	Updates dispatch and/or received dates on order.

#	Stored Procedure	R/W	API Endpoint	What It Does
				Core warehouse fulfillment tracking action.
8.4	<code>crud.usp_Order_Cancel</code>	WRITE	POST <code>/api/v1/orders/{orderId}/cancel</code>	Cancels order — sets status to Cancelled and triggers downstream notifications when patient cancels or authorization is denied.
8.5	<code>crud.usp_Order_ProductAuthStatus_Update</code>	WRITE	PUT <code>/api/v1/orders/items/{orderItemId}/authorization</code>	Updates authorization status of a product line item when authorization response is received from payer.

09 Reconciliation

6 read · 2 write

#	Stored Procedure	R/W	API Endpoint	What It Does
9.1	<code>crud.usp_Reconciliation_Payment_Add</code>	WRITE	POST <code>/api/v1/reconciliation/payments</code>	Creates a new reconciliation payment voucher — records payer payment (check/EFT) for matching against claim transactions.
9.2	<code>api.usp_Reconciliation_Payment_GetById</code>	READ	GET <code>/api/v1/reconciliation/payments/{paymentId}</code>	Single reconciliation payment voucher by ID including matched/unmatched transaction totals.
9.3	<code>api.usp_Reconciliation_OpenPayments_List</code>	READ	GET <code>/api/v1/reconciliation/payments/open</code>	All open (unmatched/partially matched) reconciliation payment vouchers. Main queue for Reconciliation workflow.
9.4	<code>api.usp_Reconciliation_Transactions_ByServiceId</code>	READ	GET <code>/api/v1/reconciliation/services/{serviceId}/unreconciled</code>	Unreconciled transactions for a specific claim service line. Shows what payments are pending matching.
9.5	<code>api.usp_Reconciliation_Transactions_ByPaymentId</code>	READ	GET <code>/api/v1/reconciliation/payments/{paymentId}/transactions</code>	All claim transactions matched to a specific payment voucher. Shows which claims the payment batch applied to.
9.6	<code>api.usp_Reconciliation_VoucherSummary_ByMonth</code>	READ	GET <code>/api/v1/reconciliation/summary/monthly</code>	Monthly summary of payment vouchers — total received, matched, and unmatched amounts. Used for

#	Stored Procedure	R/W	API Endpoint	What It Does
				month-end close reporting.
9.7	api.usp_Reconciliation_Vouchers_ByDateRange	READ	GET /api/v1/reconciliation/vouchers	All payment vouchers within a date range. Used in Reconciliation history view and audit reporting.
9.8	crud.usp_Reconciliation_BulkVoucher_Insert	WRITE	POST /api/v1/reconciliation/vouchers/bulk	Bulk-inserts multiple payment vouchers in one call when importing an ERA 835 file with payments for multiple claims/payers.

10 Warehouse / Inventory

4 read · 4 write

#	Stored Procedure	R/W	API Endpoint	What It Does
10.1	api.usp_Warehouse_Location_List	READ	GET /api/v1/warehouse/locations	All warehouse locations. Used in Warehouse Management grid and product inventory management.
10.2	crud.usp_Warehouse_Location_Upsert	WRITE	POST /api/v1/warehouse/locations	Creates a new warehouse location when a new physical storage facility or area is added.
10.3	api.usp_Product_List	READ	GET /api/v1/warehouse/products	Product catalog with paging, sorting, and search. Used in Warehouse Product Management grid and order item selection.
10.4	api.usp_Product_GetById	READ	GET /api/v1/warehouse/products/{productId}	Full product details including serial number and stock by warehouse location. Used in Product Detail view.
10.5	crud.usp_Product_Upsert	WRITE	POST /api/v1/warehouse/products	Creates or updates a product in catalog from the Warehouse Product Create and Edit forms.
10.6	crud.usp_Product_Stock_Upsert	WRITE	POST /api/v1/warehouse/products/{productId}/stock	Creates or updates stock record linking product to warehouse location with quantity and optional serial number.

#	Stored Procedure	R/W	API Endpoint	What It Does
10.7	crud.usp_Product_Stock_Transfer	WRITE	POST <code>/api/v1/warehouse/products/{productId}/stock/transfer</code>	Moves product quantity from one warehouse location to another for inventory redistribution.
10.8	api.usp_Product_QuantityByLocation_Get	READ	GET <code>/api/v1/warehouse/products/{productId}/stock/by-location</code>	Stock quantities for product broken down by warehouse location. Used in order fulfillment location selection.

11

Document Storage (AWS/S3)

0 read · 3 write

#	Stored Procedure	R/W	API Endpoint	What It Does
11.1	crud.usp_Document_PatientFile_Save	WRITE	POST <code>/api/v1/documents</code>	Creates document metadata record after S3 upload — records patient link, file type, and S3 key for document discovery.
11.2	crud.usp_Document_PatientFile_Update	WRITE	PUT <code>/api/v1/documents/{documentId}</code>	Updates document metadata — typically rename, re-categorization, or description update after upload.
11.3	crud.usp_Document_PatientFile_Delete	WRITE	DELETE <code>/api/v1/documents/{documentId}</code>	Removes document metadata record, paired with S3 object deletion handled in the application layer.

06 Security, HIPAA & Governance

OXPLiveNG holds Protected Health Information (PHI) — patient demographics, insurance coverage, diagnosis codes, and EOB/claim data. The MCP infrastructure applies defense-in-depth controls at every layer.

Control	Mechanism	Where Enforced
Least-privilege DB access	Service account <code>oxp_api_executor</code> holds only <code>EXECUTE</code> on <code>api</code> and <code>crud</code> schemas — no <code>SELECT</code> , no direct table access	SQL Server <code>GRANT</code> ; CI greps for inline SQL in API project
No direct DB from MCP	MCP Server (L3) has zero <code>Microsoft.Data.SqlClient</code> reference — enforced at project level	<code>csproj</code> dependency check in CI pipeline
PHI column flagging	Catalog marks <code>ContainsPhi=1</code> per SP; MCP redacts PHI fields unless caller holds <code>phi:read</code> scope	L2 API middleware + L3 tool response filter
Entra ID authentication	OIDC via Microsoft Entra ID for Admin Portal; <code>client_credentials</code> flow for AI agent access	L2 API + L4 Admin Portal auth middleware
HIPAA audit trail	Every tool call / API call / SP execution logged to <code>mcpadmin.AuditLog</code> with actor, target, correlation ID, and PHI-redacted details	L2 API middleware (always on, cannot be disabled)
IP whitelist	Allowed source IPs stored in DB (<code>api.usp_Admin_WhitelistIP_*</code>); checked at API gateway for non-Entra callers	L2 middleware; managed via Admin Portal (Domain 4)
Role-based access control	Tools/endpoints enabled per role; <code>IsEnabled=0</code> makes tool invisible to all callers regardless of auth	L3 MCP tool registry + L2 route authorization
Secrets management	DB connection string and API keys stored as Azure Key Vault references — never plaintext in config files	Remediates current <code>chetuSA</code> hardcoded credential (<code>Web.config:26-27</code>)
TLS 1.2+ in transit	All L2↔L3 and L3↔AI traffic over HTTPS; no HTTP fallback in production	IIS binding configuration + HSTS

07 What Gets Built

The engagement delivers four layers of infrastructure across five implementation phases:

Phase	Deliverable	Key Work
Phase 0 Foundations	DB schemas, catalog tables, service account	Create <code>api</code> and <code>crud</code> schemas. Create <code>mcpadmin.*</code> catalog database. Provision <code>exp_api_executor</code> SQL login with EXECUTE-only grant. Remediate hardcoded <code>chetuSA</code> credential via Azure Key Vault.
Phase 1 Read SPs	60 <code>api.*</code> read stored procedures	Wrap all 31 OXPLiveNG business views in explicit-column-set SPs. Add catalog rows (<code>api.SpCatalog</code> , <code>api.SpParameter</code> , <code>api.SpResultColumn</code>). Each SP is an idempotent CREATE OR ALTER in SSDT.
Phase 2 REST API	.NET 8 Web API with OpenAPI/Swagger	Routes materialized from <code>api.SpCatalog</code> — adding an SP row adds an endpoint, no code deploy. Dapper for SP calls, Serilog for structured logging, Entra ID for auth, OpenAPI contract published.
Phase 3 MCP Server + Admin Portal MVP	MCP Server (L3) + Admin Portal (L4)	MCP tools auto-generated from OpenAPI catalog. <code>stdio</code> + HTTP/SSE transports. Admin Portal with tool registry, connection manager, role management, and audit log viewer. Mirrors <code>orx-mcp-daisybill1837</code> architecture.
Phase 4 Write SPs + Full CRUD	47 <code>crud.*</code> write stored procedures	CRUD SPs for core entities (Claims, Patients, Jobs, Insurance, Reconciliation, Warehouse). CRUD generator automates boilerplate from <code>api.EntityCatalog</code> . All write SPs run inside transactions with typed <code>THROW</code> error handling.
Phase 5 Hardening	Production-ready system	Retention policies, notification rules, rate limiting, E2E test suite, HIPAA compliance review, pen-test remediation, Entra CA policy. Schedules and feature flags configured via Admin Portal.

Project Outputs

- 107 stored procedures in `api` and `crud` schemas (SSDT + migrations)
- `ExpLiveNg.Api` — .NET 8 REST API project with OpenAPI spec
- `MCPServer` + `MCPHttpServer` — .NET 8 MCP server (`stdio` + HTTP/SSE)
- `MCPAdminPortal` + `Client` — .NET 8 + React/TS admin application
- Azure Key Vault wiring for all secrets (remediates current plaintext credentials)
- CI/CD pipeline (inline SQL guard, schema validation, OpenAPI diff check)

08 Delivery Plan

Phase	Duration	Milestone Gate	Deliverables
Phase 0	Week 1	DB schemas + service account provisioned	api/crud/mcpadmin schemas · Key Vault wiring · exp_api_executor login
Phase 1	Weeks 2–4	All 58 read SPs deployed + catalog populated	58 api.* SPs · api.SpCatalog rows · SSDT project
Phase 2	Weeks 4–6	REST API live + Swagger published	OxpLiveNg.Api project · OpenAPI spec · Entra auth · Serilog audit
Phase 3	Weeks 6–9	MCP server operational + Admin Portal MVP	MCPServer · MCPHttpServer · MCPAdminPortal · claude_desktop_config.json snippet
Phase 4	Weeks 9–12	All 49 write SPs + CRUD tools live	49 crud.* SPs · CRUD generator · full tool suite (107 tools)
Phase 5	Weeks 12–16	Hardened, HIPAA-reviewed, production-ready	E2E tests · retention/notification config · pen-test sign-off · Entra CA policy

Reference implementation: The orx-mcp-daisybill1837 server (already live in ORX environment) validates this architecture and toolchain. OXPLiveNG builds on the same proven pattern with the same .NET 8 / Dapper / Serilog / ModelContextProtocol SDK stack — reducing technical risk significantly.

09 Next Steps — What We Need From You

Upon client approval of this document, work begins on Phase 0. The following items are needed from OrthoKinetix:

#	Item	Owner
1	Written approval of this document (sign below or email to Technijian project lead)	ORX
2	SQL Server access for Technijian developer account on ORX-DC-SQL-02 (10.100.106.72) — db_owner on OXPLiveNG for development, least-privilege in production	ORX IT
3	Azure tenant access for Entra ID application registration (or ORX registers the app, shares Client ID)	ORX IT / Technijian
4	Azure Key Vault access (or create new vault) for storing DB connection string and API keys	ORX IT
5	Git repo access (or create new repo in ORX Azure DevOps / GitHub organization)	ORX IT
6	IIS host and server for deploying the REST API and MCP server in development	ORX IT

10 Approval & Sign-Off

By signing below, OrthoKinetix approves the scope, architecture, and capability list defined in this document and authorizes Technijian to begin Phase 0 (Foundations). Changes to scope after approval require a written amendment.

OrthoKinetix Authorization

Approved by:

Printed Name & Title

Signature

Date

Technijian Project Lead

Acknowledged by: Rohit Jain, Technijian Inc.

Signature

Date

Questions? Contact your Technijian account team at **949.379.8499** or rjain@technijian.com

A — Domain Quick Reference

#	Domain	SPs	Scope
1	Claims	25	Claims dashboard, AR summary, transaction management, orphan matching, duplicate detection
2	Patient / Account	25	Patient demographics, insurance, medical info, service lines, guarantor, health plans, documents
3	Billing Dashboard	3	Billing job queue, patient info panel, document file-type resolution
4	Admin & Security	22	Roles, permissions, users, organizations, groups, IP whitelist, billing codes
5	Authorization	2	PA request pre-population — patient and order details for prior authorization forms
6	Documents (Core)	1	System-wide document list for document management and attachment workflows
7	Groups / Risk Payers	5	Master groups, risk payer group configuration and assignments
8	Orders	5	Order listing, detail, fulfillment tracking, cancellation, authorization status
9	Reconciliation	8	ERA/EOB payment vouchers, orphan matching, monthly close reporting, bulk ERA import
10	Warehouse / Inventory	8	Warehouse locations, product catalog, stock management, inter-location transfers
11	Document Storage (AWS/S3)	3	Patient file metadata after S3 upload — create, update, delete document records

B — Architecture Constraints (Non-Negotiable)

- **SP-only DB access (P1):** The API executes only stored procedures in `api/crud` schemas. No `SELECT/INSERT` against tables from application code.
- **API-first contract (P2):** The OpenAPI document is the contract. The MCP server is derived from it — no tool definition exists outside the catalog.
- **MCP fronts the API, never the DB (P3):** The MCP server has no SQL connection string and no `Microsoft.Data.SqlClient` reference.
- **Nothing hardcoded (P4):** Endpoints, SP mappings, tool definitions, connections, roles, schedules, and feature flags live in DB tables. Secrets live in Azure Key Vault.
- **Everything audited (P5):** Every tool call / API call / SP execution logged with actor, target, correlation ID, and PHI-redacted details.
- **Least privilege, deny-by-default (P6):** A tool/endpoint/SP is unavailable unless explicitly enabled and the caller's role permits it.

C — Companion Documents

- `docs/oxpliveng-mcp-server-specification.md` — Full 16-section system specification (v0.1, May 2026)
- `docs/oxpliveng-sp-api-mcp-catalog.md` — Catalog index + Domain 1 (Claims)
- `docs/oxpliveng-sp-api-mcp-catalog-d2-patient.md` — Domain 2 (Patient)
- `docs/oxpliveng-sp-api-mcp-catalog-d3-d6.md` — Domains 3–6 (Billing, Admin, Auth, Documents)
- `docs/oxpliveng-sp-api-mcp-catalog-d7-d11.md` — Domains 7–11 (Groups, Orders, Reconciliation, Warehouse, AWS)